

ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Покрытие кода: тестирование белого ящика

Модуль 2 · Дополнительный блок · Тест-дизайн

Ольга Лихачева · Lektion

4 курс

Факультативная тема

План блока

01 От чёрного ящика к белому: зачем смотреть в код

03 Уровни покрытия: операторы и ветви

05 MC/DC: покрытие, модифицированное по условию

07 Инструменты измерения и практика на СМП

02 Что такое покрытие кода и как оно измеряется

04 Уровни покрытия: условия и решения-условия

06 Покрытие путей и его недостижимость

08 Ограничения: сто процентов покрытия не равны корректности

От чёрного ящика к белому

Техники чёрного ящика опираются на требования и описание поведения. **Техники белого ящика** опираются на внутреннее устройство: код и его структуру.

Чёрный ящик: взгляд снаружи

Вход — требования и спецификация.

Вопрос: «что система должна делать?»

Инструмент: КЭ, ГЗ, pairwise, таблицы решений.

Не даёт ответа, все ли ветви кода задействованы.

Белый ящик: взгляд изнутри

Вход — исходный код и его структура.

Вопрос: «какие части кода ещё не выполнялись?»

Инструмент: покрытие операторов, ветвей, условий.

Дополняет чёрный ящик, а не заменяет его.

Почему это не замена: сто процентов покрытия не гарантируют корректности — тесты могут исполнять код, но ничего не проверять.

Что такое покрытие кода

Покрытие кода (code coverage) — доля структурных элементов программы, выполненных при прогоне набора тестов, выраженная в процентах.

Что считаем

Структурный элемент: оператор, ветвь, условие, путь.

На чём считаем

На результате прогона конкретного набора тестов, а не на тесте по отдельности.

Как измеряем

Инструментом: он инструментирует код и фиксирует исполненные элементы.

Формула: **покрытие = выполненные элементы ÷ всего элементов × 100%**.

Уровень покрытия задаётся выбором элемента: что считаем — то и покрываем. Поэтому «уровень покрытия» — это не одно число, а семейство метрик.

Уровень 1: покрытие операторов

Покрытие операторов (statement coverage) — доля исполняемых операторов, выполненных при прогоне набора тестов.

Самый простой и самый распространённый уровень — его по умолчанию показывают инструменты.

Показывает, есть ли код, который не исполнялся ни разу: «мёртвый» код, забытые ветви, недостижимые обработчики.

Слабое место: **не проверяет решения**. Оператор внутри `if` может исполниться, но условие при этом вычислено только в одну сторону.

Пример: `if (x > 10) { doSomething(); }` — оператор `doSomething()` покрыт, если хотя бы один тест дал `x > 10`. Что происходит при `x ≤ 10`, покрытие операторов не сообщает.

Уровень 2: покрытие ветвей

Покрытие ветвей (branch coverage, decision coverage) — доля ветвей программы, выполненных при прогоне набора тестов. Ветвь — каждый возможный результат решения.

Уровень 3: покрытие условий

Покрытие условий (condition coverage) — доля атомарных условий, вычисленных в обоих значениях: и истина, и ложь.

Атомарное условие — выражение без логических связок, например `x > 10` или `isActive`.

Ветви может удовлетворять один тест, если условий несколько: до всех ветвей можно дойти, не увидев все комбинации условий.

Условие	Тест 1	Тест 2	Оценка
<code>x > 10</code>	истина	ложь	покрыто
<code>isActive</code>	истина	ложь	покрыто

Ограничение: покрытие условий **не гарантирует** покрытие ветвей — каждое условие вычислено в обоих значениях, но конкретная их комбинация могла не встретиться.

Уровень 4: покрытие решений-условий

1

Требование 1

Сто процентов покрытия решений: каждая ветвь каждого решения выполнена.



Что даёт вместе

Устраняет слабое место покрытия условий: ветви гарантированы, как и значения всех условий.

2

Требование 2

Сто процентов покрытия условий: каждое атомарное условие вычислено в обоих значениях.



Цена

Требуется больше тестов: нужно набрать комбинации, закрывающие оба требования одновременно.

Уровень 5: MC/DC

MC/DC — покрытие решений, модифицированное по условию (modified condition/decision coverage).
Каждое атомарное условие должно **независимо** влиять на результат решения.

Идея: показать для каждого условия, что его изменение меняет результат решения при прочих равных.

Где применяют: критичное ПО — авиация (стандарт DO-178C), медицина, железнодорожная автоматика.

Тест	A	B	Решение A AND B	Роль
1	истина	истина	истина	базовый
2	ложь	истина	ложь	A влияет
3	истина	ложь	ложь	B влияет

Тесты 1 и 2 показывают влияние **A** ; тесты 1 и 3 — влияние **B** . Для двух условий достаточно трёх тестов вместо четырёх комбинаций.

Уровень 6: покрытие путей

Покрытие путей (path coverage) — доля путей через граф потока управления, выполненных при прогоне набора тестов.

Сравнение уровней покрытия

Уровень	Что требует	Включение	Применение
Операторы	каждый оператор выполнен	—	базовая метрика, по умолчанию
Ветви (решения)	каждая ветвь решения	включает операторы	рабочая норма
Условия	каждое условие в обоих значениях	не включает ветви	уточняющий уровень
Решения-условия	условия + решения вместе	включает оба	строгий уровень
MC/DC	независимое влияние каждого условия	включает предыдущий	критичное ПО
Пути	все пути через граф	включает решения-условия	недостижимо при циклах

Инструменты измерения покрытия

Как это работает

1. Инструмент инструментует код: вставляет счётчики.
2. Прогоняется набор тестов.
3. Счётчики фиксируют исполненные элементы.
4. Отчёт агрегирует покрытие по уровням.

Инструменты по языкам

Java: JaCoCo, Cobertura.

Go: встроенный `go test -cover`.

Python: coverage.py.

TypeScript/JS: Istanbul / nyc.

```
$ mvn test jacoco:report $ open target/site/jacoco/index.html Instruction coverage: 87% Branch coverage: 74% ←  
рабочая метрика
```

Сквозной пример: авторизация

Упрощённая логика авторизации на сервисе Authorization

```
public Result authorize(Card card, long amount) {  
    if (card.status != ACTIVE) {  
        return DECLINED;  
    }  
    if (amount > card.availableBalance) {  
        return DECLINED;  
    }  
    return APPROVED;  
}
```

Классы эквивалентности (чёрный ящик)

status: ACTIVE / не ACTIVE; amount: в пределах баланса / превышает. Четыре класса — четыре тест-кейса.

Ветви (белый ящик)

Четыре ветви: status $\neq$ ACTIVE, status = ACTIVE, amount > balance, amount $\leq$ balance.

Сквозной пример: покрытие ветвей

Тест	status	amount vs balance	Ожидаемый результат	Покрытые ветви
TC-01	ACTIVE	в пределах	APPROVED	status = ACTIVE, amount ≤ balance
TC-02	BLOCKED	в пределах	DECLINED	status ≠ ACTIVE
TC-03	ACTIVE	превышает	DECLINED	amount > balance

Три теста покрывают **все четыре ветви**.

Покрытие ветвей — **100%**.

Покрытие операторов тоже 100% — как следствие.

Но это **не значит**, что проверены *границы*: сумма ровно равная балансу, статус EXPIRED, отказ по лимиту. За это отвечает чёрный ящик.

Ограничения: покрытие не равно корректности

Главное ограничение: покрытие измеряет, что код *выполнялся*, а не что результат *проверялся*.

Тест без проверки: прогон кода с пустыми assertions даёт сто процентов покрытия и ноль проверенной логики.

Отсутствующее требование: в коде может не быть ветви для требования, которое просто забыли реализовать. Покрытие этого не покажет — нечего покрывать.

Комбинаторика: сто процентов ветвей не означают всех комбинаций условий.

Данные: покрытие не говорит, были ли использованы граничные и невалидные значения.

Приписывать покрытию смысл, которого в нём нет, — распространённая ошибка. Это метрика полноты исполнения, а не метрика качества проверок.

Как использовать покрытие правильно

Полезно как

Индикатор пропущенных участков кода.

Критерий для критичных модулей: покрытие ветвей как норма.

Инструмент поиска мёртвого кода.

Метрика регрессии: покрытие не должно падать.

Бесполезно как

Самоцель: гнаться за ста процентами на любом коде.

Доказательство качества: покрытие не равно корректности.

Замена тест-дизайну: без чёрного ящика набор будет неполным.

Общая метрика по проекту без разбивки по модулям.

Разумный ориентир: покрытие **ветвей** для критичных модулей, покрытие операторов как базовая проверка отсутствия мёртвого кода.

Резюме

Белый ящик опирается на структуру кода; дополняет чёрный ящик, но не заменяет его.

Покрытие кода — доля структурных элементов, исполненных при прогоне набора; измеряется инструментом.

Уровни: операторы → ветви → условия → решения-условия → MC/DC → пути. Строгость растёт, применимость падает.

Рабочая норма — покрытие ветвей: оно включает операторы и не требует недостижимых комбинаций.

MC/DC — для критичного ПО: независимое влияние каждого условия на решение.

Главное: покрытие измеряет исполнение, а не корректность. Сто процентов покрытия не означают отсутствия дефектов.

Источники

Источник	Что именно
ISTQB CTFL v4.0	Техники белого ящика: покрытие операторов и ветвей; определения уровней
ISTQB Glossary	Определения: code coverage, statement coverage, branch coverage, decision coverage, condition coverage
Copeland, гл. 4	Структурные техники тестирования, покрытие решений
Куликов, раздел 2.2	Структурные техники, покрытие и его измерение
DO-178C	Требование MC/DC для критичного авиационного ПО
JaCoCo	Практический инструмент измерения покрытия для Java

The background is a dark blue gradient. It features several glowing green lines that flow across the frame, resembling data paths or circuitry. Scattered throughout are icons of classical buildings with columns, possibly representing institutions or organizations. There are also several credit cards floating in the space, some of which are partially obscured by the glowing lines. The overall aesthetic is high-tech and digital.

Вопросы?